Malnad Gold Full Stack Application Architecture details

This document provides a comprehensive step-by-step technical explanation of all the source files running in the `backend/src` and `frontend/src` directories.

File: `backend/src/main/java/com/homestay/BackendApplication.java`

Technical Explanation

This is the main entry point of the Spring Boot application. The @SpringBootApplication annotation triggers auto-configuration, component scanning, and starts the embedded web server (Tomcat).

Source Code

```java
package com.homestay;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }

}
```

File: `backend/src/main/java/com/homestay/config/WebConfig.java`

Technical Explanation

This is a Configuration class. It contains setup configuration like CORS settings, security rules, and MVC configurations necessary for the Spring Boot application to run securely and connect properly to the frontend.

Source Code

```java
package com.homestay.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.nio.file.Path;
import java.nio.file.Paths;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:5173", "http://127.0.0.1:5173", "http://localhost:3000") // Vite defaults
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        Path uploadDir = Paths.get("uploads");
        String uploadPath = uploadDir.toFile().getAbsolutePath();

        registry.addResourceHandler("/uploads/**")
                .addResourceLocations("file:/" + uploadPath + "/");
    }
}
```

File:

`backend/src/main/java/com/homestay/controller/AuthController.java`

Technical Explanation

This is a REST Controller class. It exposes the API endpoints to the frontend, handles incoming HTTP requests (GET, POST, etc.), and returns JSON responses after interacting with repositories.

Source Code

```java
package com.homestay.controller;

import com.homestay.models.User;
import com.homestay.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Optional;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private UserRepository userRepository;

    @PostMapping("/login")
    public ResponseEntity<User> loginOrRegister(@RequestBody User loginRequest) {
        Optional<User> existingUser = userRepository.findByEmail(loginRequest.getEmail());
        if (existingUser.isPresent()) {
            User user = existingUser.get();
            // Update role if changed
            if (loginRequest.getRole() != null && !loginRequest.getRole().equals(user.getRole())) {
                user.setRole(loginRequest.getRole());
                userRepository.save(user);
            }
            return ResponseEntity.ok(user);
        } else {
            User newUser = userRepository.save(loginRequest);
            return ResponseEntity.ok(newUser);
        }
    }
}
```

File:

`backend/src/main/java/com/homestay/controller/BookingController.ja`

Technical Explanation

This is a REST Controller class. It exposes the API endpoints to the frontend, handles incoming HTTP requests (GET, POST, etc.), and returns JSON responses after interacting with repositories.

Source Code

```java
package com.homestay.controller;

import com.homestay.models.Booking;
import com.homestay.repository.BookingRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.List;

@RestController
@RequestMapping("/api/bookings")
public class BookingController {

    @Autowired
    private BookingRepository bookingRepository;

    @PostMapping
    public Booking createBooking(@RequestBody Booking booking) {
        booking.setBookingDate(LocalDateTime.now());
        booking.setPaymentStatus("COMPLETED"); // Mock success
        return bookingRepository.save(booking);
    }

    @GetMapping("/guest/{email}")
    public List<Booking> getBookingsByGuest(@PathVariable("email") String email) {
        return bookingRepository.findByGuestEmail(email);
    }

    @GetMapping("/homestay/{homestayId}")
    public List<Booking> getBookingsByHomestay(@PathVariable("homestayId") String homestayId) {
        return bookingRepository.findByHomestayId(homestayId);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> cancelBooking(@PathVariable("id") String id) {
        Booking booking = bookingRepository.findById(id).orElse(null);
        if (booking != null) {
            LocalDateTime oneDayAfter = booking.getBookingDate().plusDays(1);
            if (LocalDateTime.now().isBefore(oneDayAfter)) {
                bookingRepository.deleteById(id);
                return ResponseEntity.ok().build();
            } else {
                return ResponseEntity.badRequest().body("Cancellation period of 1 day has expired.");
            }
        }
        return ResponseEntity.notFound().build();
    }
}
```

File:

`backend/src/main/java/com/homestay/controller/FileUploadController`

Technical Explanation

This is a REST Controller class. It exposes the API endpoints to the frontend, handles incoming HTTP requests (GET, POST, etc.), and returns JSON responses after interacting with repositories.

Source Code

```java
package com.homestay.controller;

import org.springframework.core.io.Resource;
import org.springframework.core.io.UrlResource;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.util.StringUtils;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.net.MalformedURLException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

@RestController
@RequestMapping("/api/uploads")
@CrossOrigin(origins = "http://localhost:5173") // Allow React frontend
public class FileUploadController {

    private final Path fileStorageLocation;

    public FileUploadController() {
        this.fileStorageLocation = Paths.get("uploads").toAbsolutePath().normalize();
        try {
            Files.createDirectories(this.fileStorageLocation);
        } catch (Exception ex) {
            throw new RuntimeException("Could not create the directory where the uploaded files will be stored.", ex);
        }
    }

    @PostMapping
    public ResponseEntity<Map<String, String>> uploadFile(@RequestParam("file") MultipartFile file) {
        String fileName = StringUtils.cleanPath(file.getOriginalFilename());
        Map<String, String> response = new HashMap<>();

        try {
            // Check if the file's name contains invalid characters
            if (fileName.contains("..")) {
                response.put("error", "Sorry! Filename contains invalid path sequence " + fileName);
                return ResponseEntity.badRequest().body(response);
            }

            // Generate unique file name
            String extension = "";
            int i = fileName.lastIndexOf('.');
            if (i > 0) {
                extension = fileName.substring(i);
            }
            String newFileName = UUID.randomUUID().toString() + extension;

            // Copy file to the target location (Replacing existing file with the same name)
            Path targetLocation = this.fileStorageLocation.resolve(newFileName);
            Files.copy(file.getInputStream(), targetLocation, StandardCopyOption.REPLACE_EXISTING);

            String fileUrl = "http://localhost:8080/uploads/" + newFileName;
            response.put("url", fileUrl);

            return ResponseEntity.ok(response);
        } catch (IOException ex) {
            response.put("error", "Could not store file " + fileName + ". Please try again!");
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(response);
```

```java
        }
    }

    @GetMapping("/{fileName:.+}")
    public ResponseEntity<Resource> downloadFile(@PathVariable("fileName") String fileName) {
        try {
            Path filePath = this.fileStorageLocation.resolve(fileName).normalize();
            Resource resource = new UrlResource(filePath.toUri());

            if (resource.exists()) {
                // Determine content type
                String contentType = Files.probeContentType(filePath);
                if (contentType == null) {
                    contentType = "application/octet-stream";
                }

                return ResponseEntity.ok()
                        .contentType(MediaType.parseMediaType(contentType))
                        .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + resource.getFilename() + "\"")
                        .body(resource);
            } else {
                return ResponseEntity.notFound().build();
            }
        } catch (MalformedURLException ex) {
            return ResponseEntity.notFound().build();
        } catch (IOException ex) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
        }
    }
}
```

File:

`backend/src/main/java/com/homestay/controller/HomestayController.j`

Technical Explanation

This is a REST Controller class. It exposes the API endpoints to the frontend, handles incoming HTTP requests (GET, POST, etc.), and returns JSON responses after interacting with repositories.

Source Code

```java
package com.homestay.controller;

import com.homestay.models.Homestay;
import com.homestay.models.Room;
import com.homestay.repository.HomestayRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/homestays")
public class HomestayController {

    @Autowired
    private HomestayRepository homestayRepository;

    @GetMapping
    public List<Homestay> getAllHomestays(@RequestParam(name = "search", required = false) String search) {
        if (search != null && !search.isEmpty()) {
            try {
                double ratingValue = Double.parseDouble(search);
                return homestayRepository.searchHomestays(search, ratingValue);
            } catch (NumberFormatException e) {
                return homestayRepository.searchHomestaysWithoutRating(search);
            }
        }
        return homestayRepository.findAll();
    }

    @GetMapping("/admin/{email}")
    public List<Homestay> getHomestaysByAdmin(@PathVariable("email") String email) {
        return homestayRepository.findByAdminEmail(email);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Homestay> getHomestayById(@PathVariable("id") String id) {
        return homestayRepository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Homestay createHomestay(@RequestBody Homestay homestay) {
        return homestayRepository.save(homestay);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteHomestay(@PathVariable("id") String id) {
        homestayRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    // Rooms
    @PostMapping("/{homestayId}/rooms")
    public ResponseEntity<Homestay> addRoom(@PathVariable("homestayId") String homestayId, @RequestBody Room room) {
        Optional<Homestay> homestayOpt = homestayRepository.findById(homestayId);
        if (homestayOpt.isPresent()) {
            Homestay homestay = homestayOpt.get();
            homestay.getRooms().add(room);
            homestayRepository.save(homestay);
            return ResponseEntity.ok(homestay);
        }
        return ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{homestayId}/rooms/{roomId}")
```

```java
    public ResponseEntity<Homestay> deleteRoom(@PathVariable("homestayId") String homestayId, @PathVariable("roomId") String
roomId) {
        Optional<Homestay> homestayOpt = homestayRepository.findById(homestayId);
        if (homestayOpt.isPresent()) {
            Homestay homestay = homestayOpt.get();
            homestay.getRooms().removeIf(r -> r.getId().equals(roomId));
            homestayRepository.save(homestay);
            return ResponseEntity.ok(homestay);
        }
        return ResponseEntity.notFound().build();
    }
}
```

File:

`backend/src/main/java/com/homestay/controller/ReviewController.jav`

Technical Explanation

This is a REST Controller class. It exposes the API endpoints to the frontend, handles incoming HTTP requests (GET, POST, etc.), and returns JSON responses after interacting with repositories.

Source Code

```java
package com.homestay.controller;

import com.homestay.models.Homestay;
import com.homestay.models.Review;
import com.homestay.repository.HomestayRepository;
import com.homestay.repository.ReviewRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/reviews")
public class ReviewController {

    @Autowired
    private ReviewRepository reviewRepository;

    @Autowired
    private HomestayRepository homestayRepository;

    @GetMapping("/homestay/{homestayId}")
    public List<Review> getReviewsByHomestay(@PathVariable("homestayId") String homestayId) {
        return reviewRepository.findByHomestayId(homestayId);
    }

    @PostMapping
    public ResponseEntity<Review> addReview(@RequestBody Review review) {
        review.setReviewDate(LocalDateTime.now());
        Review savedReview = reviewRepository.save(review);

        // Update average rating
        List<Review> allReviews = reviewRepository.findByHomestayId(review.getHomestayId());
        double avg = allReviews.stream().mapToInt(Review::getRating).average().orElse(0.0);

        Optional<Homestay> homestayOpt = homestayRepository.findById(review.getHomestayId());
        if (homestayOpt.isPresent()) {
            Homestay h = homestayOpt.get();
            h.setAverageRating(avg);
            homestayRepository.save(h);
        }

        return ResponseEntity.ok(savedReview);
    }
}
```

File: `backend/src/main/java/com/homestay/models/Booking.java`

Technical Explanation

This is a Data Model (Entity) class. It represents the database structure using JPA annotations, mapping Java objects directly to database tables and defining relationships between them.

Source Code

```java
package com.homestay.models;

import lombok.Data;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Table;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "bookings")
public class Booking {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;
    private String homestayId;
    private String homestayName;
    private String roomId;
    private String guestEmail;

    private String guestName;
    private String guestPhone;

    private String checkInDate;
    private String checkOutDate;

    private String paymentMethod; // "ONLINE" or "CASH"
    private String paymentStatus; // "PENDING", "COMPLETED"

    private LocalDateTime bookingDate;
}
```

File: `backend/src/main/java/com/homestay/models/Homestay.java`

Technical Explanation

This is a Data Model (Entity) class. It represents the database structure using JPA annotations, mapping Java objects directly to database tables and defining relationships between them.

Source Code

```java
package com.homestay.models;

import lombok.Data;
import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;
import java.util.List;
import java.util.ArrayList;

@Data
@Entity
@Table(name = "homestays")
public class Homestay {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;
    private String name;
    private String address; // Address of the homestay
    private String locationMapUrl; // Location in map format optionally
    private String photoUrl; // Photo of the homestay from folder
    private String adminEmail; // The email of the admin who created this

    // We'll store basic details and list of rooms
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinColumn(name = "homestay_id")
    private List<Room> rooms = new ArrayList<>();

    // Derived overall rating
    private double averageRating;
}
```

File: `backend/src/main/java/com/homestay/models/Review.java`

Technical Explanation

This is a Data Model (Entity) class. It represents the database structure using JPA annotations, mapping Java objects directly to database tables and defining relationships between them.

Source Code

```java
package com.homestay.models;

import lombok.Data;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Table;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "reviews")
public class Review {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;
    private String homestayId;
    private String guestEmail;
    private String guestName;
    private int rating; // 1 to 5
    private String comment;
    private LocalDateTime reviewDate;
}
```

File: `backend/src/main/java/com/homestay/models/Room.java`

Technical Explanation

This is a Data Model (Entity) class. It represents the database structure using JPA annotations, mapping Java objects directly to database tables and defining relationships between them.

Source Code

```java
package com.homestay.models;

import lombok.Data;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Table;
import java.util.UUID;

@Data
@Entity
@Table(name = "rooms")
public class Room {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;
    private int numberOfBeds;
    private boolean isAc;
    private double price;
    private boolean hasWifi;
    private String otherFeatures;
    private String photoUrl;
}
```

File: `backend/src/main/java/com/homestay/models/User.java`

Technical Explanation

This is a Data Model (Entity) class. It represents the database structure using JPA annotations, mapping Java objects directly to database tables and defining relationships between them.

Source Code

```java
package com.homestay.models;

import lombok.Data;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Table;

@Data
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;
    private String name;
    private String email;
    private String role; // "GUEST" or "ADMIN"
    private String picture;
}
```

File:

`backend/src/main/java/com/homestay/repository/BookingRepository.ja`

Technical Explanation

This is a Repository interface. It extends Spring Data JPA's repository interfaces, providing built-in methods (CRUD operations) to interact with the database without writing manual SQL queries.

Source Code

```java
package com.homestay.repository;

import com.homestay.models.Booking;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface BookingRepository extends JpaRepository<Booking, String> {
    List<Booking> findByGuestEmail(String guestEmail);
    List<Booking> findByHomestayId(String homestayId);
}
```

File:

`backend/src/main/java/com/homestay/repository/HomestayRepository.j`

Technical Explanation

This is a Repository interface. It extends Spring Data JPA's repository interfaces, providing built-in methods (CRUD operations) to interact with the database without writing manual SQL queries.

Source Code

```java
package com.homestay.repository;

import com.homestay.models.Homestay;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface HomestayRepository extends JpaRepository<Homestay, String> {
    List<Homestay> findByNameContainingIgnoreCase(String name);
    List<Homestay> findByAdminEmail(String adminEmail);

    @Query("SELECT h FROM Homestay h WHERE lower(h.name) LIKE lower(concat('%', :search, '%')) OR lower(h.address) LIKE
lower(concat('%', :search, '%')) OR lower(h.locationMapUrl) LIKE lower(concat('%', :search, '%')) OR h.averageRating >=
:rating")
    List<Homestay> searchHomestays(@Param("search") String search, @Param("rating") Double rating);

    @Query("SELECT h FROM Homestay h WHERE lower(h.name) LIKE lower(concat('%', :search, '%')) OR lower(h.address) LIKE
lower(concat('%', :search, '%')) OR lower(h.locationMapUrl) LIKE lower(concat('%', :search, '%'))")
    List<Homestay> searchHomestaysWithoutRating(@Param("search") String search);
}
```

File:

`backend/src/main/java/com/homestay/repository/ReviewRepository.jav`

Technical Explanation

This is a Repository interface. It extends Spring Data JPA's repository interfaces, providing built-in methods (CRUD operations) to interact with the database without writing manual SQL queries.

Source Code

```java
package com.homestay.repository;

import com.homestay.models.Review;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface ReviewRepository extends JpaRepository<Review, String> {
    List<Review> findByHomestayId(String homestayId);
}
```

File:

`backend/src/main/java/com/homestay/repository/UserRepository.java`

Technical Explanation

This is a Repository interface. It extends Spring Data JPA's repository interfaces, providing built-in methods (CRUD operations) to interact with the database without writing manual SQL queries.

Source Code

```java
package com.homestay.repository;

import com.homestay.models.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, String> {
    Optional<User> findByEmail(String email);
}
```

File: `backend/src/main/resources/application.properties`

Technical Explanation

This is the main configuration properties file for Spring Boot. It configures the server port, database connection details (JDBC URL, username, password), file upload limits, and static resources.

Source Code

```
spring.application.name=backend
spring.datasource.url=jdbc:mysql://localhost:3306/homestay_booking?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=shreyas@
spring.jpa.hibernate.ddl-auto=update
server.port=8080

# File Upload Configuration
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
spring.web.resources.static-locations=file:uploads/,classpath:/META-
INF/resources/,classpath:/resources/,classpath:/static/,classpath:/public/
```

File:

`backend/src/test/java/com/homestay/BackendApplicationTests.java`

Technical Explanation

This file is part of the underlying source code supporting the core functionality of either the backend API or the frontend React rendering.

Source Code

```java
package com.homestay;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class BackendApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

File: `frontend/src/App.jsx`

Technical Explanation

This is the main React Application component. It sets up the Routing logic (using React Router), defining which page component should be rendered for each URL path, and wraps the app in the global context providers.

Source Code

```jsx
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Navbar from './components/Navbar';
import AdminPage from './pages/AdminPage';
import HomePage from './pages/HomePage';
import HomestayDetail from './pages/HomestayDetail';
import BookingsPage from './pages/BookingsPage';
import { useAuth } from './context/AuthContext';
import BackgroundSlideshow from './components/BackgroundSlideshow';

function App() {
  const { user, loading } = useAuth();

  if (loading) return <div>Loading...</div>;

  return (
    <Router>
      <BackgroundSlideshow />
      <div className="app">
        <Navbar />
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/homestay/:id" element={<HomestayDetail />} />

          <Route path="/admin" element={
            user && user.role === 'ADMIN' ? <AdminPage /> : <Navigate to="/" />
          } />

          <Route path="/bookings" element={
            user && user.role === 'GUEST' ? <BookingsPage /> : <Navigate to="/" />
          } />
        </Routes>
      </div>
    </Router>
  );
}

export default App;
```

File: `frontend/src/components/AuthModal.jsx`

Technical Explanation

This is a reusable React component. It is modular UI element (like Navbar, Modal, or Slideshow) that encapsulates its own structure, styling, and behavior, to be used inside larger pages.

Source Code

```
import React, { useState } from 'react';
import { useGoogleLogin } from '@react-oauth/google';
import axios from 'axios';
import { useAuth } from '../context/AuthContext';
import { X, User, Home } from 'lucide-react';

const AuthModal = ({ isOpen, onClose }) => {
    const { login } = useAuth();
    const [loading, setLoading] = useState(false);

    const handleGoogleLogin = useGoogleLogin({
        onSuccess: async (tokenResponse) => {
            try {
                // Fetch user info from Google using access token
                const userInfo = await axios.get('https://www.googleapis.com/oauth2/v3/userinfo', {
                    headers: { Authorization: `Bearer ${tokenResponse.access_token}` }
                });

                await login(userInfo.data, window.selectedRole);
                onClose();
            } catch (err) {
                console.error('Error fetching google user info', err);
                alert("Login failed. Please try again.");
                setLoading(false);
            }
        },
        onError: () => {
            console.error('Login Failed');
            setLoading(false);
        }
    });

    const triggerLogin = (role) => {
        setLoading(true);
        window.selectedRole = role; // Temporary ugly hack to pass role to google onSuccess
        handleGoogleLogin();
    };

    if (!isOpen) return null;

    return (
        <div className="modal-overlay">
            <div className="modal-content animate-fade-in p-6" style={{ padding: '2rem' }}>
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '1.5rem'
}}>
                    <h2>Login or Sign Up</h2>
                    <button onClick={onClose} style={{ background: 'none', border: 'none', cursor: 'pointer' }}>
                        <X size={24} />
                    </button>
                </div>

                <p style={{ marginBottom: '1.5rem', textAlign: 'center' }}>Choose your account type to proceed.</p>

                <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
                    <button
                        className="btn btn-primary"
                        style={{ padding: '1rem', fontSize: '1.1rem' }}
                        onClick={() => triggerLogin('GUEST')}
                        disabled={loading}
                    >
                        <User size={24} /> Login as Guest
                    </button>

                    <button
                        className="btn btn-secondary"
                        style={{ padding: '1rem', fontSize: '1.1rem' }}
                        onClick={() => triggerLogin('ADMIN')}
                        disabled={loading}
                    >
```

```
                        <Home size={24} /> Login as Homestay Admin
                    </button>
                </div>
            </div>
        </div>
    );
};

export default AuthModal;
```

File: `frontend/src/components/BackgroundSlideshow.jsx`

Technical Explanation

This is a reusable React component. It is modular UI element (like Navbar, Modal, or Slideshow) that encapsulates its own structure, styling, and behavior, to be used inside larger pages.

Source Code

```react
import React, { useState, useEffect } from 'react';

const images = [
  'https://images.unsplash.com/photo-1472214103451-9374bd1c798e?q=80&w=3000&auto=format&fit=crop', // Lush Valley
  'https://images.unsplash.com/photo-1542451313056-b7c8e626645f?q=80&w=3000&auto=format&fit=crop', // Mountain Lake
  'https://images.unsplash.com/photo-1469474968028-56623f02e42e?q=80&w=3000&auto=format&fit=crop', // Forest Path
  'https://images.unsplash.com/photo-1444459094776-88af74d22187?q=80&w=3000&auto=format&fit=crop', // Snowy peak / nature
];

const BackgroundSlideshow = () => {
    const [currentIndex, setCurrentIndex] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            setCurrentIndex((prev) => (prev + 1) % images.length);
        }, 3000); // 3 seconds

        return () => clearInterval(interval);
    }, []);

    return (
        <div style={{
            position: 'fixed',
            top: 0,
            left: 0,
            width: '100vw',
            height: '100vh',
            zIndex: -1,
            overflow: 'hidden'
        }}>
            {images.map((img, index) => (
                <div
                    key={img}
                    style={{
                        position: 'absolute',
                        top: 0,
                        left: 0,
                        width: '100%',
                        height: '100%',
                        backgroundImage: `url(${img})`,
                        backgroundSize: 'cover',
                        backgroundPosition: 'center',
                        transition: 'opacity 1.5s ease-in-out',
                        opacity: index === currentIndex ? 1 : 0
                    }}
                />
            ))}
            {/* Overlay to ensure the premium glass UI remains perfectly readable against bright nature photos */}
            <div style={{
                position: 'absolute',
                top: 0,
                left: 0,
                width: '100%',
                height: '100%',
                backgroundColor: 'rgba(255, 255, 255, 0.4)',
                backdropFilter: 'blur(3px)',
                WebkitBackdropFilter: 'blur(3px)'
            }} />
        </div>
    );
};

export default BackgroundSlideshow;
```

File: `frontend/src/components/Navbar.jsx`

Technical Explanation

This is a reusable React component. It is modular UI element (like Navbar, Modal, or Slideshow) that encapsulates its own structure, styling, and behavior, to be used inside larger pages.

Source Code

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import AuthModal from './AuthModal';
import { Home, LogOut, User as UserIcon } from 'lucide-react';
import { useGoogleOneTapLogin } from '@react-oauth/google';

const Navbar = () => {
    const { user, login, logout } = useAuth();
    const [isAuthModalOpen, setIsAuthModalOpen] = useState(false);
    const navigate = useNavigate();

    useGoogleOneTapLogin({
        onSuccess: async (credentialResponse) => {
            try {
                // Decode the JWT credential
                const base64Url = credentialResponse.credential.split('.')[1];
                const base64 = base64Url.replace(/-/g, '+').replace(/_/g, '/');
                const jsonPayload = decodeURIComponent(atob(base64).split('').map(function(c) {
                    return '%' + ('00' + c.charCodeAt(0).toString(16)).slice(-2);
                }).join(''));
                const decodedToken = JSON.parse(jsonPayload);

                await login(decodedToken, 'GUEST');
            } catch (err) {
                console.error("Auto login failed", err);
            }
        },
        onError: () => {
            console.log('One Tap Login Failed');
        },
        disabled: user !== null // Disable if already logged in
    });

    const handleLogout = () => {
        logout();
        navigate('/');
    };

    return (
        <>
            <nav className="navbar glass">
                <div className="container nav-container">
                    <Link to="/" className="logo">
                        <Home size={28} /> Malnad Gold Homestay
                    </Link>

                    <div className="nav-actions">
                        {user ? (
                            <>
                                {user.role === 'ADMIN' && (
                                    <Link to="/admin" className="btn btn-secondary">Admin Dashboard</Link>
                                )}
                                {user.role === 'GUEST' && (
                                    <Link to="/bookings" className="btn btn-primary">Our Bookings</Link>
                                )}
                                <div style={{ display: 'flex', alignItems: 'center', gap: '0.5rem', marginLeft: '1rem' }}>
                                    <img
                                        src={user.picture || 'https://via.placeholder.com/40'}
                                        alt={user.name}
                                        style={{ width: '40px', height: '40px', borderRadius: '50%' }}
                                    />
                                    <span style={{ fontWeight: '500' }}>{user.name}</span>
                                </div>
                                <button className="btn" onClick={handleLogout} style={{ border: '1px solid var(--danger)',
color: 'var(--danger)', marginLeft: '1rem' }}>
                                    <LogOut size={18} /> Logout
                                </button>
                            </>
```

```
                    ) : (
                        <>
                            <button className="btn btn-primary" onClick={() => setIsAuthModalOpen(true)}>
                                Login
                            </button>
                            <button className="btn btn-secondary" onClick={() => setIsAuthModalOpen(true)}>
                                Sign Up
                            </button>
                        </>
                    )}
                </div>
            </div>
        </nav>
        <AuthModal isOpen={isAuthModalOpen} onClose={() => setIsAuthModalOpen(false)} />
    </>
  );
};

export default Navbar;
```

File: `frontend/src/context/AuthContext.jsx`

Technical Explanation

This is a React Context provider. It manages global state (like user authentication status) so that any component in the application can access user details without needing props passed down manually.

Source Code

```jsx
import React, { createContext, useContext, useState, useEffect } from 'react';
import axios from 'axios';

const AuthContext = createContext();

export const useAuth = () => useContext(AuthContext);

export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        // Mock persistence
        const storedUser = localStorage.getItem('user');
        if (storedUser) {
            setUser(JSON.parse(storedUser));
        }
        setLoading(false);
    }, []);

    const login = async (googleData, role) => {
        try {
            const response = await axios.post('http://localhost:8080/api/auth/login', {
                name: googleData.name,
                email: googleData.email,
                picture: googleData.picture,
                role: role // "GUEST" or "ADMIN"
            });
            setUser(response.data);
            localStorage.setItem('user', JSON.stringify(response.data));
            return response.data;
        } catch (error) {
            console.error("Login failed", error);
            throw error;
        }
    };

    const logout = () => {
        setUser(null);
        localStorage.removeItem('user');
    };

    return (
        <AuthContext.Provider value={{ user, login, logout, loading }}>
            {!loading && children}
        </AuthContext.Provider>
    );
};
```

File: `frontend/src/index.css`

Technical Explanation

This file contains the custom CSS styles for the frontend application. It defines the layout, colors, typography, and responsive rules for giving the interface a fully styled appearance.

Source Code

```css
@import url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800&display=swap');

:root {
  /* Premium Vibrant Colors */
  --primary: #6366f1; /* Indigo 500 */
  --primary-hover: #4f46e5; /* Indigo 600 */
  --secondary: #ec4899; /* Pink 500 */
  --secondary-hover: #db2777; /* Pink 600 */
  --accent: #8b5cf6; /* Violet */
  --dark: #0f172a; /* Slate 900 */
  --dark-glass: rgba(15, 23, 42, 0.75);
  --light: #f8fafc; /* Slate 50 */
  --white: #ffffff;
  --white-glass: rgba(255, 255, 255, 0.65);
  --white-glass-strong: rgba(255, 255, 255, 0.85);
  --gray: #64748b; /* Slate 500 */
  --gray-light: #e2e8f0; /* Slate 200 */
  --danger: #ef4444;
  --danger-hover: #dc2626;

  --radius-sm: 0.5rem;
  --radius-md: 0.75rem;
  --radius-lg: 1.25rem;
  --radius-xl: 1.75rem;

  /* Modern Smooth Shadows */
  --shadow-sm: 0 2px 8px -2px rgba(99, 102, 241, 0.05);
  --shadow-md: 0 8px 16px -4px rgba(99, 102, 241, 0.1);
  --shadow-lg: 0 16px 32px -8px rgba(99, 102, 241, 0.15);
  --glow: 0 0 24px rgba(99, 102, 241, 0.35);
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Outfit', system-ui, -apple-system, sans-serif;
  color: var(--dark);
  line-height: 1.6;
  -webkit-font-smoothing: antialiased;
  min-height: 100vh;
}

@keyframes gradientBG {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}

a {
  text-decoration: none;
  color: inherit;
  transition: all 0.3s ease;
}

a:hover {
  color: var(--primary);
}

/* Typography */
h1, h2, h3, h4 {
  font-weight: 700;
  color: var(--dark);
  margin-bottom: 0.75rem;
  letter-spacing: -0.02em;
}
```

```css
p {
  color: var(--gray);
  margin-bottom: 1.25rem;
  font-size: 1.05rem;
}

/* Utilities */
.container {
  max-width: 1280px;
  margin: 0 auto;
  padding: 0 1.5rem;
}

/* Glassmorphism */
.glass {
  background: var(--white-glass);
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
  border: 1px solid rgba(255, 255, 255, 0.8);
  box-shadow: var(--shadow-sm);
}

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 0.85rem 1.75rem;
  border-radius: var(--radius-md);
  font-weight: 600;
  font-family: inherit;
  cursor: pointer;
  transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
  border: none;
  gap: 0.5rem;
  letter-spacing: 0.01em;
  font-size: 1.05rem;
}

.btn-primary {
  background: linear-gradient(135deg, var(--primary), var(--accent));
  color: var(--white);
  box-shadow: 0 4px 12px rgba(99, 102, 241, 0.3);
}

.btn-primary:hover {
  transform: translateY(-2px) scale(1.02);
  box-shadow: 0 8px 24px rgba(99, 102, 241, 0.4);
}

.btn-primary:active {
  transform: translateY(0) scale(0.98);
}

.btn-secondary {
  background: linear-gradient(135deg, var(--secondary), #f43f5e);
  color: var(--white);
  box-shadow: 0 4px 12px rgba(236, 72, 153, 0.3);
}

.btn-secondary:hover {
  transform: translateY(-2px) scale(1.02);
  box-shadow: 0 8px 24px rgba(236, 72, 153, 0.4);
}

.btn-secondary:active {
  transform: translateY(0) scale(0.98);
}
```

```
.btn-danger {
  background: linear-gradient(135deg, var(--danger), #be123c);
  color: var(--white);
  box-shadow: 0 4px 12px rgba(239, 68, 68, 0.3);
}

.btn-danger:hover {
  transform: translateY(-2px) scale(1.02);
  box-shadow: 0 8px 24px rgba(239, 68, 68, 0.4);
}

/* Cards */
.card {
  background: var(--white-glass-strong);
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
  border: 1px solid rgba(255, 255, 255, 0.9);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-md);
  overflow: hidden;
  transition: all 0.4s cubic-bezier(0.4, 0, 0.2, 1);
}

.card:hover {
  transform: translateY(-8px);
  box-shadow: var(--shadow-lg), var(--glow);
  border-color: rgba(99, 102, 241, 0.1);
}

/* Animations */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(15px) scale(0.98); }
  to { opacity: 1; transform: translateY(0) scale(1); }
}

.animate-fade-in {
  animation: fadeIn 0.6s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}

/* Forms */
.input-group {
  margin-bottom: 1.25rem;
}

.input-label {
  display: block;
  font-weight: 600;
  margin-bottom: 0.5rem;
  color: var(--dark);
  font-size: 0.95rem;
}

.input-field {
  width: 100%;
  padding: 0.85rem 1.25rem;
  background: rgba(255, 255, 255, 0.85);
  border: 2px solid var(--gray-light);
  border-radius: var(--radius-md);
  outline: none;
  transition: all 0.3s ease;
  font-family: inherit;
  font-size: 1rem;
  box-shadow: inset 0 2px 4px rgba(0,0,0,0.02);
}

.input-field:focus {
  border-color: var(--primary);
  background: var(--white);
  box-shadow: 0 0 0 4px rgba(99, 102, 241, 0.15);
}
```

```
.input-field:hover {
  border-color: #cbd5e1;
}

/* Modal */
.modal-overlay {
  position: fixed;
  inset: 0;
  background-color: rgba(15, 23, 42, 0.4);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 50;
  padding: 1.5rem;
  animation: fadeIn 0.3s ease-out;
}

.modal-content {
  background: var(--white-glass-strong);
  backdrop-filter: blur(24px);
  -webkit-backdrop-filter: blur(24px);
  border: 1px solid rgba(255, 255, 255, 0.9);
  border-radius: var(--radius-xl);
  width: 100%;
  max-width: 550px;
  max-height: 90vh;
  overflow-y: auto;
  box-shadow: var(--shadow-lg), 0 25px 50px -12px rgba(0, 0, 0, 0.25);
}

/* Navbar */
.navbar {
  position: sticky;
  top: 0;
  z-index: 40;
  padding: 1.25rem 0;
  background: rgba(255, 255, 255, 0.75);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  border-bottom: 1px solid rgba(255, 255, 255, 0.5);
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.03);
  transition: all 0.3s ease;
}

.nav-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.logo {
  font-size: 1.75rem;
  font-weight: 800;
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  background-clip: text;
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  display: flex;
  align-items: center;
  gap: 0.5rem;
  letter-spacing: -0.03em;
}

.nav-actions {
  display: flex;
  gap: 1.25rem;
  align-items: center;
```

```
}

/* Scrollbar tweaks for premium feel */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}

::-webkit-scrollbar-track {
  background: var(--light);
}

::-webkit-scrollbar-thumb {
  background: var(--gray-light);
  border-radius: 10px;
}

::-webkit-scrollbar-thumb:hover {
  background: #cbd5e1;
}
```

File: `frontend/src/main.jsx`

Technical Explanation

This is the entry point of the React frontend. It mounts the main component onto the real DOM element (div with id 'root') in the index.html file, bootstrapping the entire user interface.

Source Code

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
import { GoogleOAuthProvider } from '@react-oauth/google'
import { AuthProvider } from './context/AuthContext.jsx'

const clientId = "583570630496-6jciin0l3f4kiifjn3b1qot3mn0hhc1v.apps.googleusercontent.com";

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <GoogleOAuthProvider clientId={clientId}>
      <AuthProvider>
        <App />
      </AuthProvider>
    </GoogleOAuthProvider>
  </React.StrictMode>,
)
```

File: `frontend/src/pages/AdminPage.jsx`

Technical Explanation

This is a React Page component. It serves as a distinct view or route in the application (like Home Page, Admin Page, Booking Details), combining smaller components to form a complete screen.

Source Code

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuth } from '../context/AuthContext';
import { Plus, Trash2, MapPin } from 'lucide-react';

const AdminPage = () => {
    const { user } = useAuth();
    const [homestays, setHomestays] = useState([]);
    const [loading, setLoading] = useState(true);

    // Form state for Homestay
    const [hsForm, setHsForm] = useState({ name: '', address: '', locationMapUrl: '', photoFile: null });

    // Form state for Room
    const [roomForm, setRoomForm] = useState({ numberOfBeds: 1, isAc: false, price: '', hasWifi: true, otherFeatures: '',
photoFile: null });

    const isSuperAdmin = user?.email === 'shreyasdrshreyasdr@gmail.com';

    useEffect(() => {
        if (user) {
            fetchHomestays();
        }
    }, [user]);

    const fetchHomestays = async () => {
        try {
            const url = isSuperAdmin
                ? `http://localhost:8080/api/homestays`
                : `http://localhost:8080/api/homestays/admin/${user.email}`;
            const res = await axios.get(url);
            setHomestays(res.data);
        } catch (error) {
            console.error(error);
        } finally {
            setLoading(false);
        }
    };

    const handleCreateHomestay = async (e) => {
        e.preventDefault();
        try {
            let uploadedPhotoUrl = '';

            if (hsForm.photoFile) {
                const formData = new FormData();
                formData.append('file', hsForm.photoFile);
                const uploadRes = await axios.post('http://localhost:8080/api/uploads', formData, {
                    headers: { 'Content-Type': 'multipart/form-data' }
                });
                uploadedPhotoUrl = uploadRes.data.url;
            }

            await axios.post('http://localhost:8080/api/homestays', {
                name: hsForm.name,
                address: hsForm.address,
                locationMapUrl: hsForm.locationMapUrl,
                photoUrl: uploadedPhotoUrl,
                adminEmail: user.email // Super admins will still own what they create
            });
            setHsForm({ name: '', address: '', locationMapUrl: '', photoFile: null });
            fetchHomestays();
        } catch (error) {
            console.error(error);
        }
    };

    const handleDeleteHomestay = async (id) => {
        if (!window.confirm("Are you sure you want to delete this homestay?")) return;
```

```
        try {
            await axios.delete(`http://localhost:8080/api/homestays/${id}`);
            fetchHomestays();
        } catch (error) {
            console.error(error);
        }
    };

    const handleAddRoom = async (e, homestayId) => {
        e.preventDefault();
        try {
            let uploadedPhotoUrl = '';
            if (roomForm.photoFile) {
                const formData = new FormData();
                formData.append('file', roomForm.photoFile);
                const uploadRes = await axios.post('http://localhost:8080/api/uploads', formData, {
                    headers: { 'Content-Type': 'multipart/form-data' }
                });
                uploadedPhotoUrl = uploadRes.data.url;
            }

            await axios.post(`http://localhost:8080/api/homestays/${homestayId}/rooms`, {
                numberOfBeds: roomForm.numberOfBeds,
                isAc: roomForm.isAc,
                price: roomForm.price,
                hasWifi: roomForm.hasWifi,
                otherFeatures: roomForm.otherFeatures,
                photoUrl: uploadedPhotoUrl
            });
            setRoomForm({ numberOfBeds: 1, isAc: false, price: '', hasWifi: true, otherFeatures: '', photoFile: null });
            fetchHomestays();
        } catch (error) {
            console.error(error);
        }
    };

    const handleDeleteRoom = async (homestayId, roomId) => {
        try {
            await axios.delete(`http://localhost:8080/api/homestays/${homestayId}/rooms/${roomId}`);
            fetchHomestays();
        } catch (error) {
            console.error(error);
        }
    };

    if (loading) return <div className="container" style={{padding: '2rem'}}>Loading...</div>;

    const renderRegistrationForm = () => (
        <div className="card animate-fade-in" style={{padding: '2rem', maxWidth: '600px', margin: '0 auto', marginBottom:
'3rem'}}>
            <h2 style={{marginBottom: '1.5rem'}}>Register A New Homestay</h2>
            <form onSubmit={handleCreateHomestay}>
                <div className="input-group">
                    <label className="input-label">Homestay Name</label>
                    <input required className="input-field" value={hsForm.name} onChange={e => setHsForm({...hsForm, name:
e.target.value})} placeholder="e.g. Sunny Hills Retreat" />
                </div>
                <div className="input-group">
                    <label className="input-label">Address</label>
                    <input required className="input-field" value={hsForm.address} onChange={e => setHsForm({...hsForm,
address: e.target.value})} placeholder="e.g. 123 Main St, City" />
                </div>
                <div className="input-group">
                    <label className="input-label">Location (Map string/URL)</label>
                    <input required className="input-field" value={hsForm.locationMapUrl} onChange={e =>
setHsForm({...hsForm, locationMapUrl: e.target.value})} placeholder="e.g. 12.9716, 77.5946 or Bangalore" />
                </div>
                <div className="input-group">
                    <label className="input-label">Homestay Photo</label>
                    <input type="file" required accept="image/*" className="input-field" onChange={e => setHsForm({...hsForm,
```

```
photoFile: e.target.files[0]})} style={{paddingTop: '0.5rem'}} />
                </div>
                <button type="submit" className="btn btn-primary" style={{width: '100%', padding: '0.75rem'}}>Register
Homestay</button>
            </form>
        </div>
    );

    return (
        <div className="container" style={{padding: '2rem'}}>
            {isSuperAdmin && (
                <div style={{background: 'var(--primary)', color: 'white', padding: '1rem', borderRadius: '8px',
marginBottom: '2rem', textAlign: 'center'}}>
                    <strong>Super Admin Mode Active:</strong> You can manage all homestays in the system.
                </div>
            )}

            {(homestays.length === 0 || isSuperAdmin) && renderRegistrationForm()}

            {homestays.map((hs, index) => (
                <div key={hs.id} style={{marginBottom: '5rem', paddingBottom: '3rem', borderBottom: index < homestays.length
- 1 ? '2px solid #eee' : 'none'}}>
                    <div style={{display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'2rem'}}>
                        <h2>Profile: {hs.name}</h2>
                        <button className="btn btn-danger" onClick={() => handleDeleteHomestay(hs.id)}>
                            <Trash2 size={18}/> Remove Homestay
                        </button>
                    </div>

                    <div className="card animate-fade-in" style={{marginBottom: '3rem'}}>
                        <img src={hs.photoUrl} alt={hs.name} style={{width: '100%', height: '300px', objectFit: 'cover'}} />
                        <div style={{padding: '2rem'}}>
                            <h3 style={{fontSize: '2rem', marginBottom: '0.5rem'}}>{hs.name}</h3>
                            {hs.address && <p style={{fontSize: '1.1rem', color: 'var(--gray)', marginBottom: '1rem'}}>
{hs.address}</p>}
                            <a href={`https://www.google.com/maps/search/?
api=1&query=${encodeURIComponent(hs.locationMapUrl)}`} target="_blank" rel="noopener noreferrer" style={{display: 'flex',
alignItems: 'center', gap: '0.5rem', fontSize: '1.1rem', textDecoration: 'none', color: 'inherit'}}>
                                <MapPin size={20}/> {hs.locationMapUrl}
                            </a>
                            {isSuperAdmin && <p style={{marginTop: '0.5rem', opacity: 0.7}}>Managed by: {hs.adminEmail}</p>}
                        </div>
                    </div>

                    <div style={{display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '2rem'}}>
                        <div>
                            <h3 style={{marginBottom: '1rem'}}>Manage Rooms</h3>
                            <div className="card" style={{padding: '1.5rem'}}>
                                <h4 style={{marginBottom: '1rem'}}>Add a New Room</h4>
                                <form onSubmit={(e) => handleAddRoom(e, hs.id)}>
                                    <div className="input-group">
                                        <label className="input-label">Number of Beds</label>
                                        <input type="number" min="1" required className="input-field" value=
{roomForm.numberOfBeds} onChange={e => setRoomForm({...roomForm, numberOfBeds: parseInt(e.target.value)})} />
                                    </div>
                                    <div style={{display: 'flex', gap: '1rem', marginBottom: '1rem'}}>
                                        <label style={{display: 'flex', alignItems: 'center', gap: '0.5rem'}}>
                                            <input type="checkbox" checked={roomForm.isAc} onChange={e =>
setRoomForm({...roomForm, isAc: e.target.checked})} /> AC Room
                                        </label>
                                        <label style={{display: 'flex', alignItems: 'center', gap: '0.5rem'}}>
                                            <input type="checkbox" checked={roomForm.hasWifi} onChange={e =>
setRoomForm({...roomForm, hasWifi: e.target.checked})} /> Free Wi-Fi
                                        </label>
                                    </div>
                                    <div className="input-group">
                                        <label className="input-label">Price per Night</label>
                                        <input type="number" required className="input-field" value={roomForm.price}
onChange={e => setRoomForm({...roomForm, price: e.target.value})} placeholder="₹" />
```

```
                                    </div>
                                    <div className="input-group">
                                        <label className="input-label">Other Features</label>
                                        <input className="input-field" value={roomForm.otherFeatures} onChange={e =>
setRoomForm({...roomForm, otherFeatures: e.target.value})} placeholder="e.g. Geyser, TV, Balcony" />
                                    </div>
                                    <div className="input-group">
                                        <label className="input-label">Room Photo</label>
                                        <input type="file" required accept="image/*" className="input-field" onChange={e =>
setRoomForm({...roomForm, photoFile: e.target.files[0]})} style={{paddingTop: '0.5rem'}} />
                                    </div>
                                    <button type="submit" className="btn btn-secondary" style={{width: '100%'}}><Plus size=
{18}/> Add Room</button>
                                </form>
                            </div>
                        </div>

                        <div>
                            <h3 style={{marginBottom: '1rem'}}>Current Rooms ({hs.rooms.length})</h3>
                            {hs.rooms.map((room, idx) => (
                                <div key={room.id} className="card" style={{display: 'flex', marginBottom: '1rem', overflow:
'hidden'}}>
                                    <img src={room.photoUrl} style={{width: '120px', objectFit: 'cover'}} alt="Room" />
                                    <div style={{padding: '1rem', flex: 1}}>
                                        <h4>Room {idx + 1}</h4>
                                        <p style={{marginBottom: '0.5rem', fontSize: '0.9rem'}}>{room.numberOfBeds} Beds •
{room.isAc ? 'AC' : 'Non-AC'} • {room.hasWifi ? 'Wi-Fi' : 'No Wi-Fi'}</p>
                                        <div style={{display: 'flex', justifyContent: 'space-between', alignItems:
'center'}}>
                                            <span style={{fontWeight: 'bold', color: 'var(--primary)'}}>₹{room.price} /
night</span>
                                            <button onClick={() => handleDeleteRoom(hs.id, room.id)} className="btn btn-
danger" style={{padding: '0.25rem 0.5rem', fontSize: '0.875rem'}}>Delete</button>
                                        </div>
                                    </div>
                                </div>
                            ))}
                            {hs.rooms.length === 0 && <p className="glass" style={{padding: '1rem', borderRadius: 'var(--
radius-md)'}}>No rooms added yet. Add your first room to start accepting guests.</p>}
                        </div>
                    </div>
                </div>
            ))}
        </div>
    );
};

export default AdminPage;
```

File: `frontend/src/pages/BookingsPage.jsx`

Technical Explanation

This is a React Page component. It serves as a distinct view or route in the application (like Home Page, Admin Page, Booking Details), combining smaller components to form a complete screen.

Source Code

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuth } from '../context/AuthContext';
import { Star, Calendar, MapPin, CheckCircle } from 'lucide-react';

const BookingsPage = () => {
    const { user } = useAuth();
    const [bookings, setBookings] = useState([]);
    const [loading, setLoading] = useState(true);

    // Rating Modal State
    const [showRatingModal, setShowRatingModal] = useState(false);
    const [selectedBooking, setSelectedBooking] = useState(null);
    const [ratingForm, setRatingForm] = useState({ rating: 5, comment: '' });

    useEffect(() => {
        fetchBookings();
    }, [user]);

    const fetchBookings = async () => {
        if (!user) return;
        setLoading(true);
        try {
            const res = await axios.get(`http://localhost:8080/api/bookings/guest/${user.email}`);
            setBookings(res.data);
        } catch (error) {
            console.error(error);
        } finally {
            setLoading(false);
        }
    };

    const handleOpenRating = (booking) => {
        setSelectedBooking(booking);
        setRatingForm({ rating: 5, comment: '' });
        setShowRatingModal(true);
    };

    const submitRating = async (e) => {
        e.preventDefault();
        try {
            await axios.post('http://localhost:8080/api/reviews', {
                homestayId: selectedBooking.homestayId,
                guestEmail: user.email,
                guestName: user.name,
                rating: ratingForm.rating,
                comment: ratingForm.comment
            });
            setShowRatingModal(false);
            alert("Thank you for your review!");
        } catch (error) {
            console.error(error);
            alert("Failed to submit review.");
        }
    };

    const handleCancelBooking = async (booking) => {
        if (window.confirm("Are you sure you want to cancel this booking? This action cannot be undone.")) {
            try {
                await axios.delete(`http://localhost:8080/api/bookings/${booking.id}`);
                alert("Booking cancelled successfully.");
                fetchBookings();
            } catch (error) {
                console.error(error);
                alert(error.response?.data || "Failed to cancel booking.");
            }
        }
    };
```

```
    if (loading) return <div className="container" style={{padding: '2rem'}}>Loading bookings...</div>;

    return (
        <div className="container" style={{paddingTop: '2rem', paddingBottom: '3rem'}}>
            <h1 style={{marginBottom: '2rem', display: 'flex', alignItems: 'center', gap: '0.5rem'}}>
                <Calendar /> Our Bookings
            </h1>

            {bookings.length === 0 ? (
                <div className="card" style={{padding: '3rem', textAlign: 'center'}}>
                    <h2 style={{color: 'var(--gray)', marginBottom: '1rem'}}>No Post Bookings Yet</h2>
                    <p>When you book a homestay, it will appear here.</p>
                </div>
            ) : (
                <div style={{display: 'flex', flexDirection: 'column', gap: '1.5rem'}}>
                    {bookings.map(booking => (
                        <div key={booking.id} className="card" style={{padding: '1.5rem', display: 'flex', justifyContent:
'space-between', alignItems: 'center'}}>
                            <div>
                                <h3 style={{fontSize: '1.5rem', marginBottom: '0.5rem'}}>{booking.homestayName}</h3>
                                <p style={{color: 'var(--gray)', margin: 0, marginBottom: '0.25rem'}}>Booked on: {new
Date(booking.bookingDate).toLocaleDateString()}</p>
                                {booking.checkInDate && booking.checkOutDate && (
                                    <p style={{color: 'var(--dark)', fontWeight: '500', margin: 0}}>
                                        Stay: {new Date(booking.checkInDate).toLocaleDateString()} to {new
Date(booking.checkOutDate).toLocaleDateString()}
                                    </p>
                                )}
                                <div style={{display: 'flex', gap: '1rem', marginTop: '0.5rem', fontSize: '0.9rem', color:
'var(--gray)'}}>
                                    <span style={{display: 'flex', alignItems: 'center', gap: '0.25rem'}}><CheckCircle size=
{14} color="var(--secondary)"/> Payment {booking.paymentStatus} ({booking.paymentMethod})</span>
                                </div>
                            </div>
                            <div style={{ display: 'flex', flexDirection: 'column', gap: '0.75rem', alignItems: 'flex-end'
}}>
                                <button className="btn btn-secondary" onClick={() => handleOpenRating(booking)}>
                                    <Star size={18} /> Leave a Review
                                </button>
                                {booking.bookingDate && (new Date() - new Date(booking.bookingDate)) < (24 * 60 * 60 * 1000)
&& (
                                    <button className="btn btn-danger" onClick={() => handleCancelBooking(booking)} style={{
padding: '0.5rem 1rem', fontSize: '0.9rem' }}>
                                        Cancel Booking (Within 24h)
                                    </button>
                                )}
                            </div>
                        </div>
                    ))}
                </div>
            )}

            {/* Rating Modal */}
            {showRatingModal && (
                <div className="modal-overlay">
                    <div className="modal-content animate-fade-in p-6" style={{padding: '2rem'}}>
                        <h2 style={{marginBottom: '1.5rem'}}>Review {selectedBooking?.homestayName}</h2>
                        <form onSubmit={submitRating}>
                            <div className="input-group">
                                <label className="input-label">Rating (1 to 5)</label>
                                <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem' }}>
                                    {[1, 2, 3, 4, 5].map(num => (
                                        <button
                                            key={num}
                                            type="button"
                                            style={{
                                                background: 'none',
                                                border: 'none',
                                                cursor: 'pointer',
                                                padding: '0.5rem'
```

```
                                        }}
                                        onClick={() => setRatingForm({...ratingForm, rating: num})}
                                    >
                                        <Star size={32} fill={num <= ratingForm.rating ? "#fbbf24" : "none"} stroke={num
<= ratingForm.rating ? "#fbbf24" : "var(--gray)"} />
                                    </button>
                                ))}
                            </div>
                        </div>
                        <div className="input-group">
                            <label className="input-label">Your Comment</label>
                            <textarea
                                required
                                className="input-field"
                                rows="4"
                                value={ratingForm.comment}
                                onChange={e => setRatingForm({...ratingForm, comment: e.target.value})}
                                placeholder="How was your stay?"
                                style={{resize: 'vertical'}}
                            ></textarea>
                        </div>
                        <div style={{display: 'flex', gap: '1rem', justifyContent: 'flex-end', marginTop: '2rem'}}>
                            <button type="button" className="btn" onClick={() => setShowRatingModal(false)} style=
{{border: '1px solid var(--gray)', background: 'transparent'}}>Cancel</button>
                            <button type="submit" className="btn btn-primary">Submit Review</button>
                        </div>
                    </form>
                </div>
            </div>
        )}
    </div>
  );
};

export default BookingsPage;
```

File: `frontend/src/pages/HomePage.jsx`

Technical Explanation

This is a React Page component. It serves as a distinct view or route in the application (like Home Page, Admin Page, Booking Details), combining smaller components to form a complete screen.

Source Code

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { Link } from 'react-router-dom';
import { Search, MapPin, Star } from 'lucide-react';

const HomePage = () => {
    const [homestays, setHomestays] = useState([]);
    const [searchQuery, setSearchQuery] = useState('');
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        fetchHomestays('');
    }, []);

    const fetchHomestays = async (query) => {
        setLoading(true);
        try {
            const res = await axios.get('http://localhost:8080/api/homestays', {
                params: query ? { search: query } : {}
            });
            setHomestays(res.data);
        } catch (error) {
            console.error("Error fetching homestays", error);
        } finally {
            setLoading(false);
        }
    };

    const handleSearch = (e) => {
        e.preventDefault();
        fetchHomestays(searchQuery);
    };

    return (
        <div className="container" style={{paddingTop: '2rem', paddingBottom: '3rem'}}>
            <div style={{ textAlign: 'center', marginBottom: '3rem', paddingTop: '2rem' }}>
                <h1 style={{ fontSize: '3rem', marginBottom: '1rem', color: 'var(--primary)' }}>Find Your Perfect Stay</h1>
                <p style={{ fontSize: '1.2rem', maxWidth: '600px', margin: '0 0 2rem auto', marginLeft: 'auto', marginRight:
'auto' }}>
                    Discover top-rated homestays, cozy corners, and luxurious retreats.
                </p>

                <div className="card" style={{ padding: '1.5rem', margin: '0 auto 2rem auto', textAlign: 'left', background:
'rgba(255,255,255,0.8)' }}>
                    <h3 style={{ marginBottom: '1rem', display: 'flex', alignItems: 'center', gap: '0.5rem' }}>
                        <MapPin size={20} color="var(--primary)" /> Find Nearby Tourist Places
                    </h3>
                    <form onSubmit={(e) => {
                        e.preventDefault();

window.open(`https://www.google.com/maps/search/tourist+attractions+near+${encodeURIComponent(e.target.elements.touristLocation
'_blank');
                    }} style={{ display: 'flex', gap: '0.5rem' }}>
                        <input
                            name="touristLocation"
                            required
                            className="input-field"
                            style={{ flex: 1, padding: '0.75rem 1rem' }}
                            placeholder="Enter a city or location to explore..."
                        />
                        <button type="submit" className="btn btn-secondary" style={{ padding: '0 1.5rem', whiteSpace:
'nowrap' }}>
                            Explore with Google
                        </button>
                    </form>
                </div>

                <form onSubmit={handleSearch} style={{ display: 'flex', maxWidth: '600px', margin: '0 auto', gap: '0.5rem'
}}>
```

```
                    <div style={{ position: 'relative', flex: 1 }}>
                        <Search size={20} style={{ position: 'absolute', left: '1rem', top: '50%', transform:
'translateY(-50%)', color: 'var(--gray)' }}/>
                        <input
                            className="input-field"
                            style={{ paddingLeft: '3rem', fontSize: '1.1rem', padding: '1rem 1rem 1rem 3rem', borderRadius:
'var(--radius-xl)' }}
                            placeholder="Search by name, address, location, or rating..."
                            value={searchQuery}
                            onChange={(e) => setSearchQuery(e.target.value)}
                        />
                    </div>
                    <button type="submit" className="btn btn-primary" style={{ borderRadius: 'var(--radius-xl)', padding: '0
2rem' }}>
                        Search
                    </button>
                </form>
            </div>

            {loading ? (
                <div style={{ textAlign: 'center' }}>Loading homestays...</div>
            ) : (
                <>
                    <h2 style={{ marginBottom: '1.5rem' }}>Featured Homestays</h2>
                    {homestays.length === 0 ? (
                        <p className="glass" style={{ padding: '2rem', textAlign: 'center', borderRadius: 'var(--radius-lg)'
}}>No homestays found matching your search.</p>
                    ) : (
                        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(300px, 1fr))', gap:
'2rem' }}>
                            {homestays.map(hs => (
                                <div key={hs.id} className="card animate-fade-in" style={{ display: 'flex', flexDirection:
'column', position: 'relative' }}>
                                    <Link to={`/homestay/${hs.id}`} style={{textDecoration: 'none', color: 'inherit'}}>
                                        <img src={hs.photoUrl} alt={hs.name} style={{ width: '100%', height: '200px',
objectFit: 'cover' }} />
                                        <div style={{ padding: '1.5rem', flex: 1, display: 'flex', flexDirection: 'column'
}}>
                                            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'flex-start', marginBottom: '0.5rem' }}>
                                                <h3 style={{ fontSize: '1.25rem', margin: 0 }}>{hs.name}</h3>
                                                <div style={{ display: 'flex', alignItems: 'center', gap: '0.25rem', color:
'#fbbf24', fontWeight: 'bold' }}>
                                                    <Star size={18} fill="#fbbf24" />
                                                    <span>{hs.averageRating > 0 ? hs.averageRating.toFixed(1) : 'New'}</span>
                                                </div>
                                            </div>
                                        </div>
                                    </Link>
                                    <div style={{ padding: '0 1.5rem 1.5rem' }}>
                                        {hs.address && <p style={{ fontSize: '0.9rem', color: 'var(--gray)', margin: '0 0
0.5rem 0' }}>{hs.address}</p>}
                                        <a href={`https://www.google.com/maps/search/?
api=1&query=${encodeURIComponent(hs.locationMapUrl)}`} target="_blank" rel="noopener noreferrer" style={{display: 'flex',
alignItems: 'center', gap: '0.25rem', color: 'var(--gray)', fontSize: '0.9rem', marginBottom: '1rem', textDecoration:
'none'}}>
                                            <MapPin size={16} /> {hs.locationMapUrl}
                                        </a>
                                        <Link to={`/homestay/${hs.id}`} style={{textDecoration: 'none'}}>
                                            <div style={{ marginTop: 'auto', paddingTop: '1rem', borderTop: '1px solid var(--
gray-light)', color: 'var(--primary)', fontWeight: '500' }}>
                                                View Details →
                                            </div>
                                        </Link>
                                    </div>
                                </div>
                            ))}
                        </div>
                    )}
                </>
```

```
            )}
        </div>
    );
};

export default HomePage;
```

File: `frontend/src/pages/HomestayDetail.jsx`

Technical Explanation

This is a React Page component. It serves as a distinct view or route in the application (like Home Page, Admin Page, Booking Details), combining smaller components to form a complete screen.

Source Code

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useParams, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { MapPin, Star, Wifi, Wind, Check, X, QrCode } from 'lucide-react';

const HomestayDetail = () => {
    const { id } = useParams();
    const navigate = useNavigate();
    const { user } = useAuth();

    const [homestay, setHomestay] = useState(null);
    const [reviews, setReviews] = useState([]);
    const [loading, setLoading] = useState(true);

    // Booking Modal State
    const [showBookingModal, setShowBookingModal] = useState(false);
    const [selectedRoom, setSelectedRoom] = useState(null);
    const [bookingForm, setBookingForm] = useState({
        guestName: user?.name || '',
        guestPhone: '',
        checkInDate: '',
        checkOutDate: '',
        paymentMethod: 'UPI', // UPI, CARD, or CASH
        utrNumber: ''
    });

    const [cardDetails, setCardDetails] = useState({
        cardNumber: '',
        expiryDate: '',
        cvv: '',
        nameOnCard: ''
    });

    useEffect(() => {
        fetchHomestayDetails();
    }, [id]);

    const fetchHomestayDetails = async () => {
        setLoading(true);
        try {
            const hsRes = await axios.get(`http://localhost:8080/api/homestays/${id}`);
            setHomestay(hsRes.data);

            const reqRes = await axios.get(`http://localhost:8080/api/reviews/homestay/${id}`);
            setReviews(reqRes.data);
        } catch (error) {
            console.error(error);
        } finally {
            setLoading(false);
        }
    };

    const handleBookRoom = (room) => {
        if (!user || user.role !== 'GUEST') {
            alert("Please login as a Guest to book a room");
            return;
        }
        setSelectedRoom(room);
        setShowBookingModal(true);
    };

    const calculateTotalAmount = () => {
        if (!selectedRoom) return 0;
        if (!bookingForm.checkInDate || !bookingForm.checkOutDate) return selectedRoom.price;
        const start = new Date(bookingForm.checkInDate);
        const end = new Date(bookingForm.checkOutDate);
        const diffTime = end.getTime() - start.getTime();
        let diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
```

```
        if (diffDays <= 0) diffDays = 1;
        return selectedRoom.price * diffDays;
    };

    const confirmBooking = async (e) => {
        e.preventDefault();

        if (new Date(bookingForm.checkInDate) >= new Date(bookingForm.checkOutDate)) {
            alert("Check-out date must be after check-in date.");
            return;
        }

        if (bookingForm.paymentMethod === 'UPI') {
            if (!bookingForm.utrNumber || bookingForm.utrNumber.trim().length !== 12) {
                alert("Please complete the UPI payment and enter the valid 12-digit UTR/Reference number to confirm your
booking.");
                return;
            }
        } else if (bookingForm.paymentMethod === 'CARD') {
            if (!cardDetails.cardNumber || cardDetails.cardNumber.replace(/\D/g, '').length < 15) {
                alert("Please enter a valid card number.");
                return;
            }
            if (!cardDetails.expiryDate || !cardDetails.cvv || !cardDetails.nameOnCard) {
                alert("Please fill in all card details.");
                return;
            }
            // Mock card processing time
            alert("Processing Card Payment...");
        }

        try {
            await axios.post('http://localhost:8080/api/bookings', {
                homestayId: homestay.id,
                homestayName: homestay.name,
                roomId: selectedRoom.id,
                guestEmail: user.email,
                guestName: bookingForm.guestName,
                guestPhone: bookingForm.guestPhone,
                checkInDate: bookingForm.checkInDate,
                checkOutDate: bookingForm.checkOutDate,
                paymentMethod: bookingForm.paymentMethod
            });
            setShowBookingModal(false);
            alert("Booking Confirmed! View it in 'Our Bookings'");
            navigate('/bookings');
        } catch (error) {
            console.error(error);
            alert("Failed to book room");
        }
    };

    if (loading) return <div className="container" style={{ padding: '2rem' }}>Loading details...</div>;
    if (!homestay) return <div className="container" style={{ padding: '2rem' }}>Homestay not found.</div>;

    return (
        <div className="container" style={{ paddingTop: '2rem', paddingBottom: '3rem' }}>
            <div className="card animate-fade-in" style={{ marginBottom: '3rem', overflow: 'hidden' }}>
                <img src={homestay.photoUrl} alt={homestay.name} style={{ width: '100%', height: '400px', objectFit: 'cover'
}} />
                <div style={{ padding: '2rem' }}>
                    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'1rem' }}>
                        <h1 style={{ fontSize: '2.5rem', margin: 0 }}>{homestay.name}</h1>
                        <div style={{ display: 'flex', alignItems: 'center', gap: '0.5rem', fontSize: '1.25rem', color:
'#fbbf24', fontWeight: 'bold' }}>
                            <Star fill="#fbbf24" /> {homestay.averageRating > 0 ? homestay.averageRating.toFixed(1) : 'New'}
                        </div>
                    </div>
                    {homestay.address && <p style={{ fontSize: '1.2rem', color: 'var(--gray)', marginBottom: '1rem' }}>
```

```
{homestay.address}</p>}
                    <a href={`https://www.google.com/maps/search/?
api=1&query=${encodeURIComponent(homestay.locationMapUrl)}`} target="_blank" rel="noopener noreferrer" style={{ display:
'flex', alignItems: 'center', gap: '0.5rem', fontSize: '1.2rem', color: 'var(--gray)', textDecoration: 'none', marginBottom:
'1.5rem' }}>
                        <MapPin />  {homestay.locationMapUrl}
                    </a>
                    <a href=
{`https://www.google.com/maps/search/tourist+attractions+near+${encodeURIComponent(homestay.locationMapUrl)}`}
target="_blank" rel="noopener noreferrer" className="btn btn-secondary" style={{ display: 'inline-flex', alignItems:
'center', gap: '0.5rem', textDecoration: 'none' }}>
                        <MapPin size={18} />  View Nearby Tourist Attractions
                    </a>
                </div>
            </div>

            <div style={{ display: 'grid', gridTemplateColumns: '2fr 1fr', gap: '2rem' }}>
                <div>
                    <h2 style={{ marginBottom: '1.5rem' }}>Available Rooms</h2>
                    <div style={{ display: 'flex', flexDirection: 'column', gap: '1.5rem' }}>
                        {homestay.rooms.length === 0 && <p>No rooms currently available.</p>}
                        {homestay.rooms.map((room, idx) => (
                            <div key={room.id} className="card" style={{ display: 'flex', overflow: 'hidden' }}>
                                <img src={room.photoUrl} alt="Room" style={{ width: '200px', objectFit: 'cover' }} />
                                <div style={{ padding: '1.5rem', flex: 1, display: 'flex', flexDirection: 'column' }}>
                                    <h3 style={{ fontSize: '1.25rem', marginBottom: '0.5rem' }}>Room {idx + 1}
({room.numberOfBeds} Beds)</h3>

                                    <div style={{ display: 'flex', gap: '1rem', marginBottom: '1rem', color: 'var(--gray)'
}}>
                                        {room.isAc && <span style={{ display: 'flex', alignItems: 'center', gap: '0.25rem'
}}><Wind size={16} />  AC</span>}
                                        {room.hasWifi && <span style={{ display: 'flex', alignItems: 'center', gap: '0.25rem'
}}><Wifi size={16} />  Wi-Fi</span>}
                                    </div>
                                    <p style={{ fontSize: '0.9rem', marginBottom: '1rem' }}>{room.otherFeatures}</p>

                                    <div style={{ marginTop: 'auto', display: 'flex', justifyContent: 'space-between',
alignItems: 'center' }}>
                                        <span style={{ fontSize: '1.5rem', fontWeight: 'bold', color: 'var(--primary)' }}>₹
{room.price}<span style={{ fontSize: '1rem', color: 'var(--gray)' }}>/night</span></span>
                                        <button className="btn btn-primary" onClick={() => handleBookRoom(room)}>Book This
Room</button>
                                    </div>
                                </div>
                            </div>
                        ))}
                    </div>
                </div>

                <div>
                    <h2 style={{ marginBottom: '1.5rem' }}>Guest Reviews</h2>
                    <div class="card" style={{ padding: '1.5rem' }}>
                        {reviews.length === 0 ? (
                            <p style={{ color: 'var(--gray)' }}>No reviews yet.</p>
                        ) : (
                            <div style={{ display: 'flex', flexDirection: 'column', gap: '1.5rem' }}>
                                {reviews.map(review => (
                                    <div key={review.id} style={{ borderBottom: '1px solid var(--gray-light)', paddingBottom:
'1rem' }}>
                                        <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom:
'0.5rem' }}>
                                            <strong>{review.guestName}</strong>
                                            <div style={{ display: 'flex', color: '#fbbf24' }}><Star size={14} fill="#fbbf24"
/> {review.rating}</div>
                                        </div>
                                        <p style={{ fontSize: '0.9rem', margin: 0 }}>{review.comment}</p>
                                    </div>
                                ))}
                            </div>
```

```
                        )}
                    </div>
                </div>
            </div>

            {/* Booking Modal */}
            {showBookingModal && (
                <div className="modal-overlay">
                    <div className="modal-content animate-fade-in p-6" style={{ padding: '2rem' }}>
                        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'1.5rem' }}>
                            <h2>Book Room in {homestay.name}</h2>
                            <button onClick={() => setShowBookingModal(false)} style={{ background: 'none', border: 'none',
cursor: 'pointer' }}><X size={24} /></button>
                        </div>
                        <div style={{ marginBottom: '1.5rem', padding: '1rem', backgroundColor: 'var(--light)', borderRadius:
'var(--radius-md)' }}>
                            <p style={{ margin: 0, fontSize: '1.1rem' }}>Total Price: <strong>₹{calculateTotalAmount()}
</strong></p>
                            {bookingForm.checkInDate && bookingForm.checkOutDate && (
                                <p style={{ margin: 0, marginTop: '0.25rem', fontSize: '0.9rem', color: 'var(--gray)' }}>
                                    ₹{selectedRoom?.price} × {Math.max(1, Math.ceil((new
Date(bookingForm.checkOutDate).getTime() - new Date(bookingForm.checkInDate).getTime()) / (1000 * 60 * 60 * 24)))} night(s)
                                </p>
                            )}
                        </div>

                        <form onSubmit={confirmBooking}>
                            <div className="input-group">
                                <label className="input-label">Full Name</label>
                                <input required className="input-field" value={bookingForm.guestName} onChange={e =>
setBookingForm({ ...bookingForm, guestName: e.target.value })} />
                            </div>
                            <div className="input-group">
                                <label className="input-label">Phone Number</label>
                                <input required type="tel" className="input-field" value={bookingForm.guestPhone} onChange={e
=> setBookingForm({ ...bookingForm, guestPhone: e.target.value })} />
                            </div>
                            <div className="input-group">
                                <label className="input-label">Email Address</label>
                                <input disabled className="input-field" value={user.email} style={{ backgroundColor: 'var(--
gray-light)' }} />
                            </div>

                            <div style={{ display: 'flex', gap: '1rem', marginBottom: '1rem' }}>
                                <div className="input-group" style={{ flex: 1, marginBottom: 0 }}>
                                    <label className="input-label">Check-in Date</label>
                                    <input required type="date" min={new Date().toISOString().split('T')[0]}
className="input-field" value={bookingForm.checkInDate} onChange={e => setBookingForm({ ...bookingForm, checkInDate:
e.target.value })} />
                                </div>
                                <div className="input-group" style={{ flex: 1, marginBottom: 0 }}>
                                    <label className="input-label">Check-out Date</label>
                                    <input required type="date" min={bookingForm.checkInDate || new
Date().toISOString().split('T')[0]} className="input-field" value={bookingForm.checkOutDate} onChange={e => setBookingForm({
...bookingForm, checkOutDate: e.target.value })} />
                                </div>
                            </div>

                            <div className="input-group">
                                <label className="input-label">Payment Method</label>
                                <select className="input-field" value={bookingForm.paymentMethod} onChange={e =>
setBookingForm({ ...bookingForm, paymentMethod: e.target.value })}>
                                    <option value="UPI">Online Payment (UPI/Apps)</option>
                                    <option value="CARD">Credit/Debit Card</option>
                                    <option value="CASH">Cash on Arrival</option>
                                </select>
                            </div>

                            {bookingForm.paymentMethod === 'UPI' && (
```

```
                                <div style={{ textAlign: 'center', marginBottom: '1.5rem', padding: '1.5rem',
backgroundColor: '#f0f4f8', borderRadius: '16px', boxShadow: 'inset 0 2px 4px rgba(0,0,0,0.05)' }}>

                                    {/* Direct App Links */}
                                    <div style={{ marginBottom: '1.5rem' }}>
                                        <p style={{ fontWeight: '600', marginBottom: '0.75rem', color: '#1f2937' }}>Pay
Directly Using Apps</p>
                                        <div style={{ display: 'flex', gap: '1rem', justifyContent: 'center', flexWrap:
'wrap' }}>
                                            <a href={`tez://upi/pay?
pa=shreyasdr.gowda25@oksbi&pn=Shreyas%20DR&cu=INR&am=${calculateTotalAmount()}`} onClick={(e) => {
                                                // If on desktop, this link won't do anything, so we simulate the flow for
testing
                                                e.preventDefault();
                                                alert("Simulating redirect to Google Pay...");
                                                setTimeout(() => {
                                                    setBookingForm({ ...bookingForm, utrNumber: '123456789012' });
                                                    alert("Test Payment Successful! UTR has been auto-filled.");
                                                }, 1000);
                                            }} className="btn" style={{ background: '#fff', color: '#333', border: '1px solid
#ddd', padding: '0.75rem 1rem', display: 'flex', alignItems: 'center', gap: '0.5rem', borderRadius: '12px', boxShadow: '0 2px
5px rgba(0,0,0,0.05)', flex: 1, minWidth: '120px', justifyContent: 'center', textDecoration: 'none' }}>
                                                <img
src="https://upload.wikimedia.org/wikipedia/commons/f/f2/Google_Pay_Logo.svg" alt="GPay" style={{ height: '18px' }} /> GPay
                                            </a>
                                            <a href={`phonepe://pay?
pa=shreyasdr.gowda25@oksbi&pn=Shreyas%20DR&cu=INR&am=${calculateTotalAmount()}`} onClick={(e) => {
                                                // Simulate for testing
                                                e.preventDefault();
                                                alert("Simulating redirect to PhonePe...");
                                                setTimeout(() => {
                                                    setBookingForm({ ...bookingForm, utrNumber: '987654321098' });
                                                    alert("Test Payment Successful! UTR has been auto-filled.");
                                                }, 1000);
                                            }} className="btn" style={{ background: '#fff', color: '#5f259f', border: '1px
solid #ddd', padding: '0.75rem 1rem', display: 'flex', alignItems: 'center', gap: '0.5rem', borderRadius: '12px', boxShadow:
'0 2px 5px rgba(0,0,0,0.05)', flex: 1, minWidth: '120px', justifyContent: 'center', textDecoration: 'none' }}>
                                                <h4 style={{ margin: 0, color: '#5f259f' }}>PhonePe</h4>
                                            </a>
                                        </div>
                                        <p style={{ marginTop: '0.5rem', fontSize: '0.85rem', color: '#6b7280' }}>(Clicking
these buttons will simulate a payment for testing)</p>
                                    </div>

                                    <div style={{ position: 'relative', display: 'flex', alignItems: 'center',
justifyContent: 'center', margin: '1rem 0' }}>
                                        <div style={{ flex: 1, height: '1px', background: '#d1d5db' }}></div>
                                        <span style={{ padding: '0 10px', color: '#6b7280', fontSize: '0.9rem', fontWeight:
'500' }}>OR SCAN QR</span>
                                        <div style={{ flex: 1, height: '1px', background: '#d1d5db' }}></div>
                                    </div>

                                    {/* Scanner Recreated from Image */}
                                    <div style={{ background: '#fff', padding: '1.5rem', borderRadius: '24px', boxShadow: '0
10px 25px rgba(0,0,0,0.08)', margin: '0 auto', maxWidth: '300px' }}>
                                        <div style={{ display: 'flex', justifyContent: 'center', alignItems: 'center', gap:
'0.5rem', marginBottom: '1rem' }}>
                                            <div style={{ width: '32px', height: '32px', borderRadius: '50%', background:
'#65a30d', color: '#fff', display: 'flex', alignItems: 'center', justifyContent: 'center', fontWeight: 'bold', fontSize:
'1.1rem' }}>S</div>
                                            <span style={{ fontSize: '1.25rem', fontWeight: '500', color: '#1f2937'
}}>Shreyas DR</span>
                                        </div>

                                        <div style={{ position: 'relative', marginBottom: '1rem' }}>
                                            <img
                                                src={`https://api.qrserver.com/v1/create-qr-code/?
size=300x300&data=upi%3A%2F%2Fpay%3Fpa%3Dshreyasdr.gowda25%40oksbi%26pn%3DShreyas%2520DR%26cu%3DINR%26am%3D${calculateTotalAmour
                                                alt="UPI QR Code"
                                                style={{ width: '100%', height: 'auto', borderRadius: '12px', display:
```

```
'block' }}
                                            />
                                            {/* Center Google Pay Logo over QR */}
                                            <div style={{ position: 'absolute', top: '50%', left: '50%', transform:
'translate(-50%, -50%)', background: '#fff', borderRadius: '50%', padding: '6px', width: '40px', height: '40px', display:
'flex', alignItems: 'center', justifyContent: 'center', boxShadow: '0 2px 4px rgba(0,0,0,0.1)' }}>
                                                <img
src="https://upload.wikimedia.org/wikipedia/commons/f/f2/Google_Pay_Logo.svg" alt="GPay" style={{ width: '24px' }} />
                                            </div>
                                        </div>

                                        {/* AMOUNT ADDED TO SCANNER */}
                                        <div style={{ background: '#fef3c7', padding: '0.75rem', borderRadius: '12px',
marginBottom: '1rem' }}>
                                            <span style={{ fontSize: '0.9rem', color: '#92400e', textTransform: 'uppercase',
fontWeight: 'bold', letterSpacing: '0.05em' }}>Amount Payable</span>
                                            <h2 style={{ margin: '0.25rem 0 0 0', color: '#b45309', fontSize: '1.8rem' }}>₹
{calculateTotalAmount()}</h2>
                                        </div>

                                        <p style={{ marginTop: '0.5rem', fontSize: '0.95rem', color: '#4b5563',
letterSpacing: '0.02em' }}>UPI ID: shreyasdr.gowda25@oksbi</p>
                                    </div>
                                    <p style={{ marginTop: '1.5rem', fontSize: '1rem', color: '#4b5563', fontWeight: '500'
}}>Scan the QR code with any UPI app to pay</p>

                                    <div style={{ marginTop: '1.5rem', textAlign: 'left', borderTop: '1px solid #e5e7eb',
paddingTop: '1.25rem' }}>
                                        <label className="input-label" style={{ color: '#166534', fontWeight: 'bold',
display: 'flex', alignItems: 'center', gap: '0.5rem', marginBottom: '0.75rem' }}>
                                            <Check size={18} /> Confirm Payment
                                        </label>
                                        <input
                                            required={bookingForm.paymentMethod === 'UPI'}
                                            type="text"
                                            minLength={12}
                                            maxLength={12}
                                            placeholder="Enter 12-Digit UTR/Reference No."
                                            className="input-field"
                                            value={bookingForm.utrNumber}
                                            onChange={e => setBookingForm({ ...bookingForm, utrNumber:
e.target.value.replace(/\D/g, '').slice(0, 12) })}
                                            style={{ borderColor: '#22c55e', backgroundColor: '#f0fdf4' }}
                                        />
                                    </div>
                                </div>
                            )}

                            {bookingForm.paymentMethod === 'CARD' && (
                                <div style={{ marginBottom: '1.5rem', padding: '1.5rem', backgroundColor: '#f8fafc',
borderRadius: '12px', border: '1px solid #e2e8f0' }}>
                                    <h3 style={{ marginBottom: '1rem', fontSize: '1.1rem' }}>Enter Card Details</h3>
                                    <div className="input-group">
                                        <label className="input-label">Card Number</label>
                                        <input required={bookingForm.paymentMethod === 'CARD'} type="text" placeholder="0000
0000 0000 0000" maxLength="19" className="input-field" value={cardDetails.cardNumber} onChange={(e) => {
                                            let val = e.target.value.replace(/\D/g, '');
                                            val = val.replace(/(\d{4})/g, '$1 ').trim();
                                            setCardDetails({ ...cardDetails, cardNumber: val });
                                        }} />
                                    </div>
                                    <div style={{ display: 'flex', gap: '1rem' }}>
                                        <div className="input-group" style={{ flex: 1 }}>
                                            <label className="input-label">Expiry Date</label>
                                            <input required={bookingForm.paymentMethod === 'CARD'} type="text"
placeholder="MM/YY" maxLength="5" className="input-field" value={cardDetails.expiryDate} onChange={(e) => {
                                                let val = e.target.value.replace(/\D/g, '');
                                                if (val.length > 2) val = val.substring(0, 2) + '/' + val.substring(2);
                                                setCardDetails({ ...cardDetails, expiryDate: val });
                                            }} />
```

```
                                        </div>
                                        <div className="input-group" style={{ flex: 1 }}>
                                            <label className="input-label">CVV</label>
                                            <input required={bookingForm.paymentMethod === 'CARD'} type="password"
placeholder="•••" maxLength="4" className="input-field" value={cardDetails.cvv} onChange={(e) => setCardDetails({
...cardDetails, cvv: e.target.value.replace(/\D/g, '') })} />
                                        </div>
                                    </div>
                                    <div className="input-group" style={{ marginBottom: 0 }}>
                                        <label className="input-label">Name on Card</label>
                                        <input required={bookingForm.paymentMethod === 'CARD'} type="text" placeholder="John
Doe" className="input-field" value={cardDetails.nameOnCard} onChange={(e) => setCardDetails({ ...cardDetails, nameOnCard:
e.target.value })} />
                                    </div>
                                </div>
                            )}

                            <button type="submit" className="btn btn-secondary" style={{ width: '100%', padding: '1rem',
fontSize: '1.1rem' }}>Pay ₹{calculateTotalAmount()} & Confirm Booking</button>
                        </form>
                    </div>
                </div>
            )}
        </div>
    );
};

export default HomestayDetail;
```